

Actividad: Server with MySQL2 TS

Materia: Programación de sistemas web.

Profesor: Frank Elgar Delgado Araiza.

Grado y grupo: ISC 5ª.

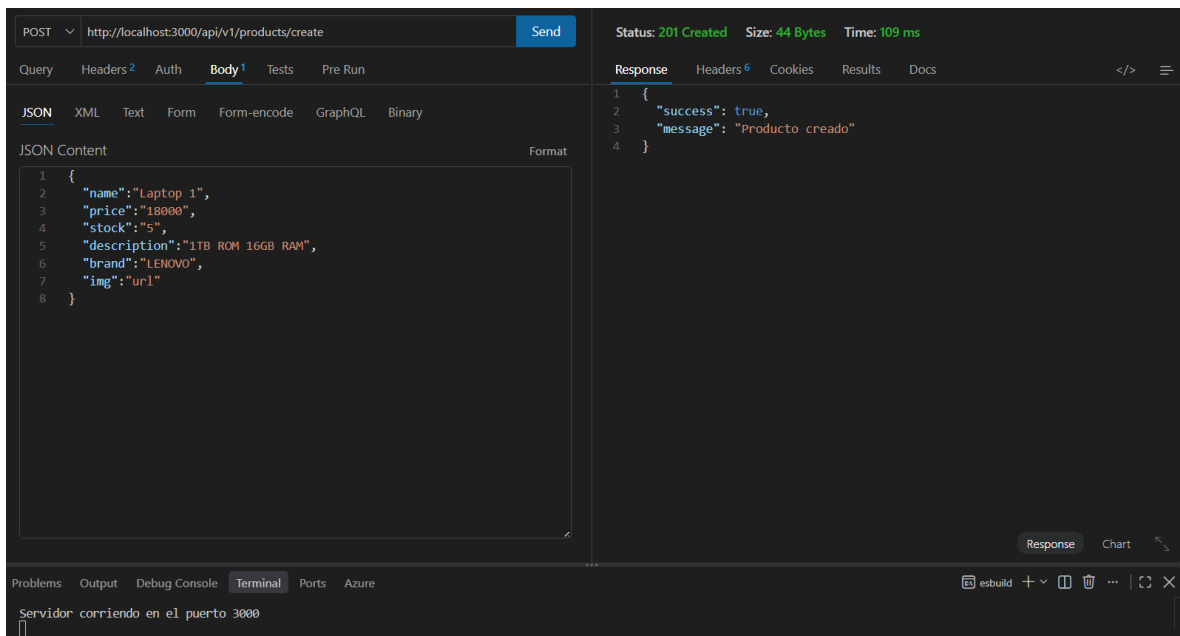
Alumno: Vázquez Delgado Cristian de Jesús

ID: 548662

Fecha: 8/10/2026

Evidencia de entrega.

1. Creación de productos.



The screenshot shows the VS Code REST Client interface. The top bar indicates a POST request to `http://localhost:3000/api/v1/products/create` with a status of `201 Created`, size of `44 Bytes`, and time of `126 ms`. The left pane shows the request body in JSON format:

```
1 {
2   "name": "Laptop 2",
3   "price": "25999",
4   "stock": "2",
5   "description": "2TB ROM 16GB RAM",
6   "brand": "AZUS",
7   "img": "url"
8 }
```

The right pane shows the response in JSON format:

```
1 {
2   "success": true,
3   "message": "Producto creado"
4 }
```

The bottom status bar shows the terminal with the text `Servidor corriendo en el puerto 3000`.

The screenshot shows the VS Code REST Client interface. The top bar indicates a POST request to `http://localhost:3000/api/v1/products/create` with a status of `201 Created`, size of `44 Bytes`, and time of `127 ms`. The left pane shows the request body in JSON format:

```
1 {
2   "name": "Laptop 3",
3   "price": "10500",
4   "stock": "8",
5   "description": "500GB ROM 16GB RAM",
6   "brand": "GATEWAY",
7   "img": "url"
8 }
```

The right pane shows the response in JSON format:

```
1 {
2   "success": true,
3   "message": "Producto creado"
4 }
```

The bottom status bar shows the terminal with the text `Servidor corriendo en el puerto 3000`.

2. consulta general y consulta por ID de un producto activo.

GET `http://localhost:3000/api/v1/products/getAll` Send

Query Headers ² Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1
```

Response Headers ⁶ Cookies Results Docs </> ≡

```
2  "success": true,
3  "data": [
4    {
5      "id": 1,
6      "name": "Laptop 1",
7      "price": "18000.00",
8      "stock": 5,
9      "description": "1TB ROM 16GB RAM",
10     "brand": "LENOVO",
11     "img": "url",
12     "active": 1
13   },
14   {
15     "id": 2,
16     "name": "Laptop 2",
17     "price": "25999.00",
18     "stock": 2,
19     "description": "2TB ROM 16GB RAM",
20     "brand": "AZUS",
21     "img": "url",
22     "active": 1
23   },
24   {
25     "id": 3,
26     "name": "Laptop 3",
```

Problems Output Debug Console **Terminal** Ports Azure

Servidor corriendo en el puerto 3000

GET `http://localhost:3000/api/v1/products/getById/2` Send

Query Headers ² Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1
```

Response Headers ⁶ Cookies Results Docs </> ≡

```
1  {
2    "success": true,
3    "data": {
4      "id": 2,
5      "name": "Laptop 2",
6      "price": "25999.00",
7      "stock": 2,
8      "description": "2TB ROM 16GB RAM",
9      "brand": "AZUS",
10     "img": "url",
11     "active": 1
12   }
13 }
```

Problems Output Debug Console **Terminal** Ports Azure

Servidor corriendo en el puerto 3000

3. actualización completa y su comprobación.

The screenshot shows the VS Code interface with a REST client extension. The top bar indicates a PUT request to `http://localhost:3000/api/v1/products/update/1`. The 'Body' tab is selected, showing a JSON payload for updating a product. The 'Response' tab shows a successful 200 OK status with a message in Spanish. The bottom terminal shows the server is running on port 3000.

```
PUT http://localhost:3000/api/v1/products/update/1
{
  "name": "Laptop PRO MAX",
  "price": "19999",
  "stock": "4",
  "description": "2TB ROM 8GB RAM",
  "brand": "LENOVO CORPORATION",
  "img": "new-url"
}
```

Status: 200 OK Size: 49 Bytes Time: 120 ms

```
{
  "success": true,
  "message": "Producto actualizado"
}
```

Terminal: Servidor corriendo en el puerto 3000

The screenshot shows the VS Code interface with a REST client extension. The top bar indicates a GET request to `http://localhost:3000/api/v1/products/getById/1`. The 'Body' tab is selected, showing an empty JSON payload. The 'Response' tab shows a successful 200 OK status with the full product details. The bottom terminal shows the server is running on port 3000.

```
GET http://localhost:3000/api/v1/products/getById/1
{
}
```

Status: 200 OK Size: 173 Bytes Time: 26 ms

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Laptop PRO MAX",
    "price": "19999.00",
    "stock": 4,
    "description": "2TB ROM 8GB RAM",
    "brand": "LENOVO CORPORATION",
    "img": "new-url",
    "active": 1
  }
}
```

Terminal: Servidor corriendo en el puerto 3000

4. Cambio de precio de un producto creado y su comprobación.

The screenshot shows the VS Code REST Client interface. The top bar indicates a PATCH request to `http://localhost:3000/api/v1/products/changePrice/1` with a status of 200 OK, size of 47 Bytes, and time of 112 ms. The 'Body' tab is selected, showing a JSON request:

```
{  "price": "18999.99"}
```

. The 'Response' tab shows the server's reply:

```
{  "success": true,  "message": "Precio actualizado"}
```

. The bottom status bar shows 'Servidor corriendo en el puerto 3000'.

The screenshot shows the VS Code REST Client interface. The top bar indicates a GET request to `http://localhost:3000/api/v1/products/getById/1` with a status of 200 OK, size of 173 Bytes, and time of 25 ms. The 'Body' tab is selected, showing an empty JSON request. The 'Response' tab shows the server's reply:

```
{  "success": true,  "data": {    "id": 1,    "name": "Laptop PRO MAX",    "price": "18999.99",    "stock": 4,    "description": "2TB ROM 8GB RAM",    "brand": "LENOVO CORPORATION",    "img": "new-url",    "active": 1  }  }
```

. The bottom status bar shows 'Servidor corriendo en el puerto 3000'.

5. Baja lógica y comprobación de que el producto dado de baja ya no aparece en `getAll` ni en `getById/:id`;

DELETE

Query Headers ² Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1
```

Status: 200 OK Size: 47 Bytes Time: 110 ms

Response Headers ⁶ Cookies Results Docs

```
1 {
2   "success": true,
3   "message": "Producto eliminado"
4 }
```

Problems Output Debug Console **Terminal** Ports Azure

esbuild +

Servidor corriendo en el puerto 3000

```
[]
```

GET

Query Headers ² Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1
```

Status: 200 OK Size: 284 Bytes Time: 17 ms

Response Headers ⁶ Cookies Results Docs

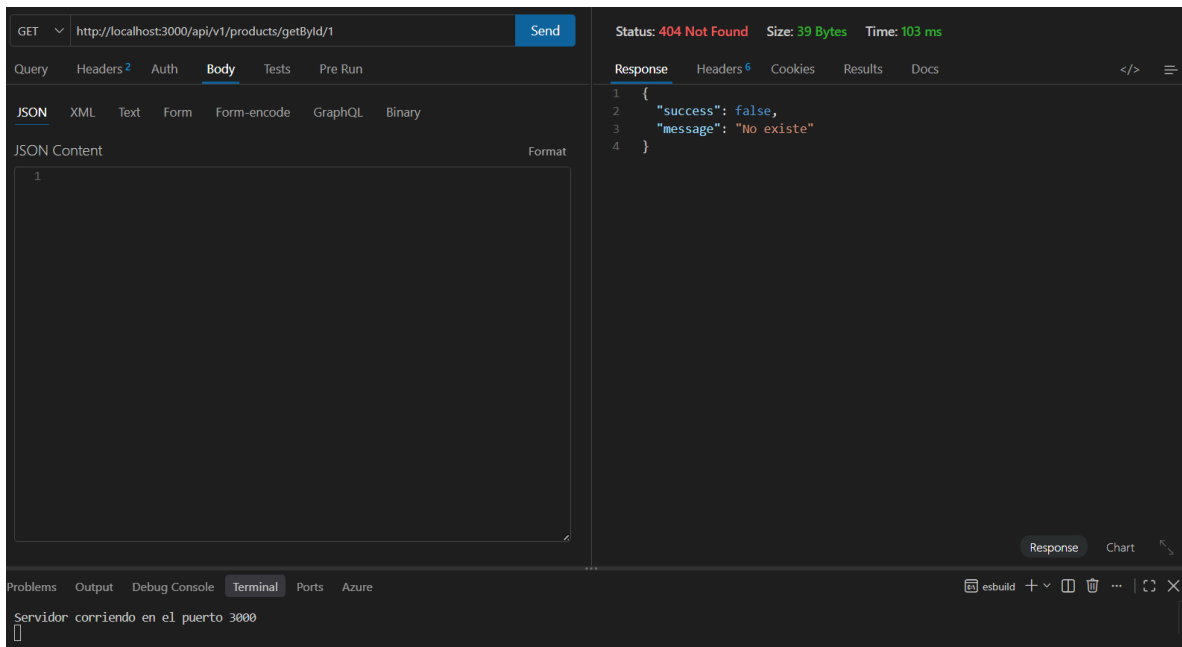
```
1 {
2   "success": true,
3   "data": [
4     {
5       "id": 2,
6       "name": "Laptop 2",
7       "price": "25999.00",
8       "stock": 2,
9       "description": "2TB ROM 16GB RAM",
10      "brand": "AZUS",
11      "img": "url",
12      "active": 1
13    },
14    {
15      "id": 3,
16      "name": "Laptop 3",
17      "price": "10500.00",
18      "stock": 8,
19      "description": "500GB ROM 16GB RAM",
20      "brand": "GATEWAY",
21      "img": "url",
22      "active": 1
23    }
24  ]
25 }
```

Problems Output Debug Console **Terminal** Ports Azure

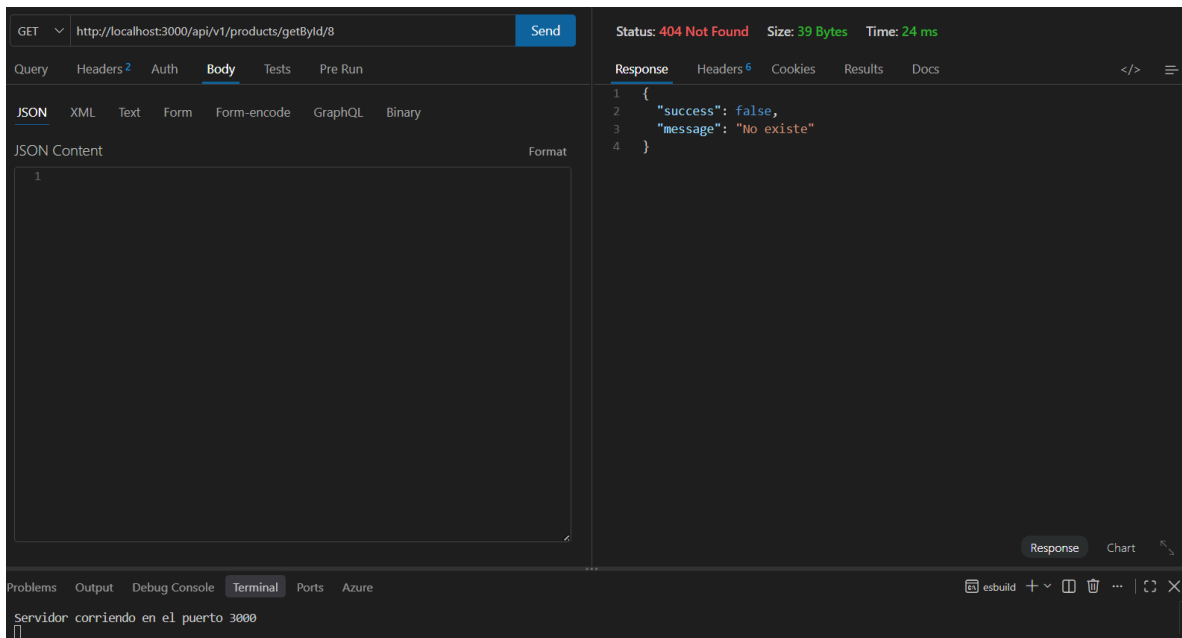
esbuild +

Servidor corriendo en el puerto 3000

```
[]
```



6. Caso de ID inexistente.



7. Caso de datos inválidos (id con una letra).

GET

http://localhost:3000/api/v1/products/getById/A

Send

Query

Headers 2

Auth

Body

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1

Status: 400 Bad Request

Size: 41 Bytes

Time: 18 ms

Response

Headers 6

Cookies

Results

Docs

</>

≡

1

{

2

"success": false,

3

"message": "ID invalido"

4

}

Response

Chart

Problems

Output

Debug Console

Terminal

Ports

Azure

esbuild

+

⌵

□

⌵

⋮

⌵

⌵

⌵

⌵

×

Servidor corriendo en el puerto 3000

⌵